

DATA 583

Advanced Predictive Modelling



Generalized Linear Models

- **Maximum Likelihood Estimation in the Exponential Family**
- **Weighted Nonlinear Least-Squares and Generalized Linear Models**
- **Deviance**
- **Types of Residuals**
- **A Bootstrap Approach to Model-Checking**
- **Quasilielihood and Overdispersion**

Maximum likelihood estimation — exponential family

We begin by fitting an exponential family model $f(y; \theta)$ when there is a single response value y . The log likelihood is given by

$$\ell(\theta) = \log(f(y; \theta)) = \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi).$$

The maximum likelihood estimator for θ can be obtained by differentiating with respect to θ

$$\ell'(\theta) = \frac{y - b'(\theta)}{a(\phi)}$$

and finding the solution of $\ell'(\theta) = 0$. In other words, we must solve the equation

$$b'(\theta) = y$$

for θ .

Estimation from samples of observations

- When we have multiple response values, $y_1, y_2, \dots, y_n$ where each $y_j \sim f(y_j; \theta_j)$, we can set up the log likelihood for each data point.

- We find that the maximum likelihood estimator for θ_j is

$$\hat{\theta}_j = b'^{-1}(y_j)$$

for $j = 1, 2, \dots, n$.

- This type of model for the response values is referred to as a saturated model.
- The saturated model is very flexible and allows the closest fit to the data. When this model does not fit the data well, the implication is that the assumed underlying probability model is incorrect.

Reducing model complexity through modelling with covariates

- Inclusion of covariate information allows us to reduce the number of parameters in the model, while providing some predictive power.
- At a minimum, we seek estimators with
 1. low bias.
 2. small variance (i.e. we seek efficiency).
 3. consistency: the estimator converges to the true parameter in probability when sample size goes to infinity.
- In fact, the next sequence of examples illustrate how maximum likelihood estimation in the exponential family is just a disguised form of weighted nonlinear least-squares estimation.

A normal model via best linear unbiased estimation

- Consider the regression through the origin model

$$y_j = \beta_1 x_j + \varepsilon_j$$

where ε_j is normally distributed with mean 0 and variance σ^2 .

- canonical parameter $\theta_j = \beta_1 x_j$, with $b(\theta_j) = \frac{1}{2}\theta_j^2$.
- $\hat{\theta}_j = y_j$, for $j = 1, 2, \dots, n$.
- $\hat{\beta}_1 = y_j/x_j$ for $j = 1, 2, \dots, n$.

A normal model via best linear unbiased estimation

- A simple way to combine the above estimates into a single estimate would be to take the average. This would give us an estimator of the form

$$\hat{\beta}_1 = \sum_{j=1}^n \frac{1}{nx_j} y_j,$$

- a linear combination of the y 's:

$$\hat{\beta}_1 = \sum_{j=1}^n a_j y_j = a^\top y \tag{1}$$

which is unbiased, and has the smallest possible variance among all such estimators.

A normal model via best linear unbiased estimation

-

$$E[\hat{\beta}_1] = \sum_{j=1}^n a_j E[y_j] = \beta_1 \sum_{j=1}^n a_j x_j = \beta_1$$

provided that $\sum_{j=1}^n a_j x_j = 1$.

-

$$\text{Var}(\hat{\beta}_1) = \sum_{j=1}^n a_j^2 \text{Var}(y_j) = \sigma^2 \sum_{j=1}^n a_j^2.$$

Here, we have used the assumption that all of the responses have the same variance, σ^2 and independent.

- In order to obtain the smallest possible variance for the estimator of β_1 , we must solve the optimization problem:

$$\min_a a^\top a : \quad a^\top x = 1.$$

Optimal estimation of β_1

Lagrange multipliers can be used to solve this problem. The Lagrangian is

$$L = a^\top a + \lambda(a^\top x - 1).$$

Differentiating with respect to a and setting to 0, we have

$$2a + \lambda x = 0. \tag{2}$$

Pre-multiplying by $a^\top$ and using the constraint, we obtain

$$2a^\top a + \lambda = 0$$

which tells us that $\lambda = -2a^\top a$.

Equation (2) can be used again:

$$2a = -\lambda x.$$

Optimal estimation of β_1

The squared norm of each side is

$$4a^\top a = \lambda^2 x^\top x$$

which, when λ is replaced by the expression given earlier, gives

$$4a^\top a = 4(a^\top a)^2 x^\top x.$$

Simplifying this tells us that

$$a^\top a = \frac{1}{x^\top x}.$$

This allows us to express λ fully in terms of x : $\lambda = \frac{-2}{x^\top x}$. Plugging this into (2) and solving for a leads to

$$a = \frac{x}{x^\top x}$$

and plugging this into (1) gives

A normal model via best linear unbiased estimation

-

$$\hat{\beta}_1 = \frac{\sum_{j=1}^n x_j y_j}{\sum_{j=1}^n x_j^2}. \quad (3)$$

- This estimator is the least-squares estimator for the slope in the regression through the origin model.
- What we have proved here is a version of the Gauss-Markov theorem: least-squares estimation leads to best linear unbiased estimation in simple regression with independent observations having constant variance.

Example – Poisson regression through the origin

- We suppose y_j has a Poisson distribution with parameter $\theta_j = x_j\beta$. The log likelihood expressed as an exponential family model is

$$\ell_j = y_j\beta x_j - b(\beta x_j) + c(y_j, \phi)$$

where $a(\phi) = 1$, and $b(\theta_j) = \mathbf{e}^{\theta_j}$.

- Differentiating with respect to β gives the estimating equation

$$y_j x_j - b'(\beta x_j) x_j = y_j x_j - \mathbf{e}^{\beta x_j} x_j = 0. \quad (4)$$

- The MLE of β is then $\hat{\beta} = \frac{\log(y_j)}{x_j}$.

Example – Poisson regression through the origin

If we have observations $y_1, y_2, \dots, y_n$, we are again confronted with the question as to how to combine estimates of the above form to obtain a single good estimate.

Taking the cue from the previous (normal error) example where the least-squares estimator was best possible, we re-consider the estimating equations (4) from the standpoint of least-squares.

However, because the equation is nonlinear and because the variance of y_j is nonconstant (i.e. $\text{Var}(y_j) = e^{\beta x_j}$) we must use weighted nonlinear least-squares.

Recall that the weights are the inverses of the variances.

The weighted least-squares problem is to minimize

$$\sum_{j=1}^n (y_j - e^{\beta x_j})^2 e^{-\beta x_j}$$

with respect to β .

Example – Poisson regression through the origin

- We will need to assume a value of β in order to calculate the weights.
- Then we need to practically solve a sequence of minimizations of the form

$$\sum_{j=1}^n (y_j - \mathbf{e}^{\beta x_j})^2 \mathbf{e}^{-\beta^* x_j}$$

- Do this by differentiating the objective function with respect to β and solving the estimating equation

$$\sum_{j=1}^n (y_j - \mathbf{e}^{\beta x_j}) x_j \mathbf{e}^{(\beta - \beta^*) x_j} = 0 \quad (5)$$

Example – Poisson regression through the origin

- Repeatedly assign $\hat{\beta}$ to β^* until the original β^* and updated $\hat{\beta}$ coincide.
- The numerical computation eventually becomes equivalent to solving

$$\sum_{j=1}^n x_j (y_j - \mathbf{e}^{\beta x_j}) = 0$$

which is exactly what would be solved if we maximize the likelihood.

Example – Poisson regression through the origin

- **Normally, we would run this procedure, updating the weights each time, until we achieve convergence to within a given tolerance.**
- **The above technique of iterating the weighted nonlinear least-squares with updated weights at each stage is called iteratively reweighted least-squares or IRLS.**

Example – Poisson regression through the origin

- We consider the ACF data set for 3 time points as an example.
- We employ one iteration of the weighted nonlinear least-squares, with a starting guess of $\beta^* = 1$.

```
source("AC1.R") # ACF in Section 1, with 1 injection
betastar <- 1 # starting guess
y.nls <- nls(y ~ exp(beta*x), start =
  list(beta = betastar), weights = exp(-betastar*x),
  data = AC1)
coef(y.nls)

##          beta
## 0.5828984
```

Example – Poisson regression through the origin

- Now, we need to update the weights using the estimate of β , 0.5828984, as a starting value:

```
y.nls <- nls(y ~ exp(beta*x),
             start = list(beta = coef(y.nls)),
             weights = exp(-coef(y.nls)*x), data = AC1)
coef(y.nls)

##          beta
## 0.5369971
```

- Note that the resulting estimate is quite close to the first one.

Example – Poisson regression through the origin

- Four more runs with the latest value of β gives convergence:

```
for (i in 1:4) {
y.nls <- nls(y ~ exp(beta*x),
  start = list(beta = coef(y.nls)),
  weights = exp(-coef(y.nls)*x), data = AC1)
if (i > 2) print(coef(y.nls))
}
```

```
##      beta
## 0.5322843
## beta.beta
## 0.5322843
```

Poisson regression through the origin - MLE

- The log likelihood based on n independent observations becomes

$$\ell(\beta) = \sum_{j=1}^n y_j \beta x_j - \sum_{j=1}^n b(\beta x_j) + \sum_{j=1}^n c(y_j, \phi)$$

- Differentiating this with respect to β gives

$$\frac{\partial \ell}{\partial \beta} = \sum_{j=1}^n (y_j x_j - b'(\beta x_j) x_j)$$

- Estimating equation for the MLE for β as

$$\frac{\partial \ell}{\partial \beta} = \sum_{j=1}^n x_j (y_j - \mathbf{e}^{\beta x_j}) = 0.$$

Poisson regression through the origin - MLE

- Compare the estimating equations obtained by weighted nonlinear least-squares (slide 13) and for maximum likelihood estimation on the previous slide. They are the same.



- The maximum likelihood estimator is numerically equivalent to the weighted nonlinear least-squares estimator, and iteratively re-weighted least-squares can be used to do the calculation.

Example - AC1 data via MLE

- Using the `glm()` function, with the `poisson` family, we can estimate the Poisson regression through the origin model for the AC1 data as follows:

```
y.glm <- glm(y ~ x - 1, family = poisson, data = AC1)
coef(y.glm)
```

```
##           x
## 0.5322843
```

- The value of the estimate is identical to the one obtained earlier, using weighted nonlinear least-squares.

Multiple covariates: canonical link function - Poisson case

- Since we set $\theta = \mathbf{X}\beta$, we were using the canonical link function in our model.

- The exponential family form of the Poisson distribution is

$$\ell(\lambda) = y \log(\lambda) - \lambda - \log(y!)$$

-

$$\theta = \log(\lambda) = \mathbf{X}\beta$$

is the canonical link function.

- λ is the mean of Poisson distribution, that is $\log(\mu) = \mathbf{X}\beta$.

Canonical link function - normal case

- The exponential family form of the normal distribution is

$$\ell(\lambda) = (y\mu - \frac{1}{2}\mu^2)/\sigma^2 + c(y, \phi)$$

-

$$\theta = \mu = \mathbf{X}\beta$$

is the canonical link function.

- This link function is the identity function.

Canonical link function - binomial case

- For a binomial random variable y_j having mean $\mu_j = p_j$, the log likelihood, written in the form of the exponential family, is

$$\ell_j(\theta_j) = y_j\theta_j - b(\theta_j)$$

where $\theta_j = \log(p_j/(1 - p_j))$, and $b(\theta_j) = \log(1 + e^{\theta_j})$.

- The logit function is the canonical link function

$$b'(\theta_j) = \frac{e^{\theta_j}}{1 + e^{\theta_j}}$$

and

$$b''(\theta_j) = \frac{e^{\theta_j}}{(1 + e^{\theta_j})^2}.$$

Implementation when the link function is canonical

We are given n independent observations on a response variable y that follows an exponential family distribution, together with corresponding observations on k predictor variables $x_1, x_2, \dots, x_k$.

- Likelihood function:

$$\ell_j(\theta_j) = \frac{y_j \theta_j - b(\theta_j)}{a(\phi)} + c(y_j, \phi)$$

- Using the canonical link function

$$\theta_j = \mathbf{X}_j \boldsymbol{\beta}$$

where $\mathbf{X}_j$ denotes the j th row of the design matrix $\mathbf{X}$. The vector $\boldsymbol{\beta}$ contains the regression coefficients.

Implementation when the link function is canonical

- Minimize the weighted nonlinear least-squares objective

$$\sum_{j=1}^n (y_j - b'(\theta_j))^2 / \mathbf{Var}(y_j)$$

with respect to the coefficients of β .

- Using $\theta_j = \mathbf{X}_j\beta$

$$\sum_{j=1}^n (y_j - b'(\mathbf{X}_j\beta))^2 / b''(\mathbf{X}_j\beta).$$

- We can try fairly arbitrary weights as a starting guess, and then solve a succession of weighted nonlinear least-squares problems where the weights are updated at each stage.

Implementation when the link function is canonical

- At stage m , we assume that we have an estimate $\hat{\beta}^*$. To obtain the $(m + 1)$ st updated estimate of $\hat{\beta}$, we apply nonlinear least-squares to minimize

$$\sum_{j=1}^n (y_j - b'(\mathbf{X}_j\beta))^2 / b''(\mathbf{X}_j\beta^*)$$

with respect to β . When the minimizer is within a prescribed tolerance of β^* , we declare convergence, and return the current value of β as the estimate.

Example – R code for binary data with logit link

- The weighted nonlinear least-squares objective function to be solved iteratively is

$$\sum_{j=1}^n (y_j - \mathbf{e}^{\beta_0 + \beta_1 x_j} / (1 + \mathbf{e}^{\beta_0 + \beta_1 x_j}))^2 / b''(\beta_0^* + x_j \beta_1^*).$$

Example – R code for binary data with logit link

The following function implements one step of the iteration:

```
logitstep <- function(x, y, bstar) {  
  logit.nls <- nls(y ~ exp(b0 + b1*x) / (1+exp(b0+b1*x)),  
    start = c(b0 = bstar[1], b1 = bstar[2]),  
    weights = (1+exp(bstar[1]+bstar[2]*x))^2 /  
              exp(bstar[1] + bstar[2]*x))  
  coef(logit.nls)  
}
```

Example – R code for binary data with logit link

To test the function, we consider 20 observations of simulated data from a model where $\text{logit}(p) = 1 + 2x$, where the x 's are uniformly distributed on the interval $(-3, 2)$. :

```
x <- runif(20, min = -3, max = 2)
y <- rbinom(20, size = 1, prob =
exp(1 + 2*x) / (1 + exp(1 + 2*x)))
```

Example – R code for binary data with logit link

Running one iteration of the nonlinear least-squares with a starting value of (0,0) gives

```
betastar <- c(0, 0)
betaout <- logitstep(x, y, bstar = betastar)
betaout

##           b0           b1
## 1.121990  2.409553
```

Running a second iteration, with the output from the previous iteration as the starting value, we have

```
betastar <- as.numeric(betaout)
betaout <- logitstep(x, y, bstar = betastar)
betaout

##           b0           b1
## 0.9380631  2.4019293
```


Example – R code for binary data with logit link

The following code could be used to iterate the method until the updated estimate is within 10^{-7} of the input estimate:

```
while (sqrt(sum((betaout - betastar)^2)) > 1e-7) {  
  betastar <- as.numeric(betaout)  
  betaout <- logitstep(x, y, bstar = betastar)  
}  
betaout  
  
##           b0           b1  
## 0.9468647 2.4362765
```

Implementation when the link function is not canonical

- When the function linking the mean of the responses to the covariates is not the canonical link function,

$$g(\mu_j) = \mathbf{X}_j\beta$$

the inverse of the link function is used in place of the inverse of the derivative of $b(\theta)$

- The m th step of the IRLS algorithm requires minimizing

$$\sum_{j=1}^n (y_j - g^{-1}(\mathbf{X}_j\beta))^2 / b''(\theta_j^*) \quad (6)$$

where $\theta_j^* = b'^{-1}(\mu_j^*) = b'^{-1}(g^{-1}(\mathbf{X}_j\beta^*))$.

Example – Bernoulli with log link

- For a binomial random variable y_j having mean $\mu_j = p_j$, the log likelihood, written in the form of the exponential family, is

$$\ell_j(\theta_j) = y_j\theta_j - b(\theta_j)$$

where $\theta_j = \log(p_j/(1 - p_j))$, and $b(\theta_j) = \log(1 + e^{\theta_j})$.

-

$$b''(\theta_j) = \frac{e^{\theta_j}}{(1 + e^{\theta_j})^2}.$$

- Use link function:

$$g(\mu_j) = \log(\mu_j) = \mathbf{X}_j\beta$$

Example – Bernoulli with log link

Note that $g^{-1}(\mathbf{X}_j\beta) = \mathbf{e}^{(\mathbf{X}_j\beta)}$ so the weighted nonlinear least-squares objective function to be solved iteratively is

$$\sum_{j=1}^n \frac{(y_j - \mathbf{e}^{\mathbf{X}_j\beta})^2}{\mathbf{e}^{\theta_j^*} / (1 + \mathbf{e}^{\theta_j^*})^2}$$

where $\theta_j^* = \text{logit}(\mathbf{e}^{\mathbf{X}_j\beta^*})$

Statistical inference for generalized linear models

- A model with fewer parameters than the saturated model is called a reduced model.

- The (scaled) deviance is defined as

$$2(\ell_{\text{saturated}} - \ell_{\text{reduced model}}).$$

- The deviance measures the decrease in the likelihood that accrues from considering the reduced model.
- Note that the deviance is a log likelihood ratio for a test where one would be checking whether the saturated model is more appropriate than the reduced model.

Deviance

The null deviance is the deviance when the reduced model is the simplest possible model, i.e. where there are no covariates, only an intercept.

The residual deviance is the deviance when the reduced model is a particular model involving covariates.

The deviance and the null deviance can be extracted from the model object directly using the `summary()` function.

```
library(MPV)
p13.5$inc1000 <- p13.5$x1/1000
y.logit <- glm(y ~ inc1000 + x2, data = p13.5,
              family = binomial(link = "logit"))
summary(y.logit)$deviance

## [1] 21.08152

summary(y.logit)$null.deviance

## [1] 27.72589
```

Deviance - example

- The AC1 data include counts y of lesions in part of a rat colon at one of three possible incubation times x .
- Poisson regression for AC1 data, including intercept:

```
y.glm0 <- glm(y ~ x, data = AC1, family = poisson)
summary(y.glm0)$deviance

## [1] 59.83603

summary(y.glm0)$null.deviance

## [1] 59.9127
```

Deviance and sum of squares

- For the normal linear regression model, if the residual deviance is multiplied by ϕ , the dispersion parameter, we recover the residual sum of squares

$$2(\ell_{\text{saturated}} - \ell_{\text{reduced model}})\phi = \sum (y_i - x_i\hat{\beta})^2$$

- The null deviance multiplied by ϕ corresponds to the total (corrected) sum of squares.

The residual deviance — asymptotic χ^2 distribution

- The residual deviance has an asymptotic chi-squared distribution for some non-normal models, for example, the Poisson regression model.
- The expected value of a χ^2 random variable is equal to its degrees of freedom, a basic model check consists of comparing the residual deviance to its degrees of freedom
- If the residual deviance is much greater than its degrees of freedom, it is unlikely that the model is adequate for the data.
- The asymptotics do not work for the binary logistic regression model.

Example - AC1 data

- **There are 22 observations.**
- **the model involved 1 regression parameter, so there were 21 residual degrees of freedom.**
- **In this case, the residual deviance is larger than its expected value, providing evidence that the model might not be adequate.**

Example - coal fracture data

- The data frame `p13.7` in the *MPV* package contains 44 observations on fractures of coal seams in a collection of Virginia coal mines.
- Measurements on four other variables are included in the data frame.
- We fit a Poisson regression model to the entire data set as follows:

```
coal.glm2 <- glm(y ~ ., data = p13.7, family = poisson)
```

Example - coal fracture data

The residual deviance is

```
summary(coal.glm2)$deviance
```

```
## [1] 37.91796
```

- There are 44 observations.
- the model involved 5 regression parameters, so there were 39 residual degrees of freedom.
- In this case, the residual deviance is very similar to its expected value, providing evidence that the model might be adequate - but further graphical diagnostics are needed to check the model.

Example - Poisson regression for cigarette data

- For the cigarette butt data of the *MPV* package, we can fit a Poisson regression model to relate expect cigarette butt count to distance from smoking gazebo, as follows:

```
cig.glm <- glm(count ~ distance, data = cigbutts,  
  family = poisson)  
summary(cig.glm)$coefficients
```

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	3.553514259	0.1735102026	20.48015	3.236877e-93
##	distance	-0.001695636	0.0002008638	-8.44172	3.127010e-17

Example - Poisson regression for cigarette data

- The residual deviance is

```
summary(cig.glm)$deviance # (scaled deviance since phi=1)  
## [1] 9.003466
```

- Here, we have fit a model with $p = 2$ parameters and there are 15 observations. Thus, the scaled deviance is slightly less than expected (i.e. $n - p = 13$).

Example - missile success-failure data

- We consider the data of `p13.1` from the *MPV* package here, concerning successes and failures of missile firing. A logistic regression model is fit as follows:

```
missile.glm <- glm(y ~ x, data = p13.1, family = binomial)
summary(missile.glm)$coefficients
```

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	6.0708839	2.108996265	2.878566	0.003994883
##	x	-0.0177047	0.006075513	-2.914107	0.003567073

Example - missile success-failure - deviance coincidence

- Next, calculate the deviance:

```
summary(missile.glm)$deviance
```

```
## [1] 20.36366
```

- Here, we have fit a logistic regression model with $p = 2$ parameters and there are 25 observations.
- There is a relatively close match between the scaled deviance here and the residual degrees of freedom, but this is somewhat coincidental, as shown in the next example, involving simulated data.

Example - simulated Bernoulli data - deviance failure

- Here we simulate 100 observations from the model

$$p(x) = \frac{e^{x+2}}{1 + e^{x+2}}$$

where the x values are randomly uniformly distributed on the interval $(0, 1)$:

```
n <- 100
x <- runif(100)
y <- rbinom(n, 1, exp(x+2) / (1+exp(x+2)))
sim.df <- data.frame(x = x, y = y); rm(x, y)
```

Example - simulated Bernoulli data - deviance failure

- Fitting the logistic model (to estimate the slope and intercept parameters) is as follows:

```
sim.glm <- glm(y ~ x, data = sim.df, family = binomial)
summary(sim.glm)$coefficients
```

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	2.4664904	0.8107146	3.0423659	0.002347264
##	x	0.5634248	1.4295973	0.3941143	0.693496629

- The parameter estimates are not far from the truth ($\beta_0 = 2, \beta_1 = 1$).

Example - simulated Bernoulli data - deviance failure

- If the asymptotic χ^2 distribution on $n - p = 98$ degrees of freedom was reasonable, we would expect the scaled deviance to be near 98. Instead it is calculated as

```
summary(sim.glm)$deviance
```

```
## [1] 45.23824
```

- This shows clearly that the asymptotics do not work for the Bernoulli case.

Analysis of Deviance is a new form of ANOVA

- It is also possible to set up approximate tests and confidence intervals for generalized linear models in much the same way as for normal regression
- Analysis of variance tests based on χ^2 statistics on $p - r$ degrees of freedom and resulting F statistics on $p - r$ and $n - (p - 1)$ could be used to test the null hypothesis that the reduced model is true versus the alternative that some or all of the other coefficients were nonzero.

Example - fractures in coal seams

- Recall `p13.7` which has 5 variables, including counts of fractures, `y`
- Fit a model relating `y` to `x1` and `x2`, and then to all variables. Test whether any of the remaining variables further explain the response, using the Analysis of Deviance.

```
coal.glm <- glm(y ~ x1+x2, data = p13.7, family = poisson)
anova(coal.glm, coal.glm2)
```

```
## Analysis of Deviance Table
##
## Model 1: y ~ x1 + x2
## Model 2: y ~ x1 + x2 + x3 + x4
##      Resid. Df Resid. Dev Df Deviance
## 1          41      42.094
## 2          39      37.918  2    4.1758
```

Example - does income matter when deciding to buy a car?

Earlier, we fit a model to predict whether an individual with a given income and a car of a certain age would be likely to purchase a car within 6 months, using the `p13.5` data set.

In order to test if income should be added in to a model that already contains the other variable, we need to fit

```
y.logit2 <- glm(y ~ x2, data = p13.5,
               family = binomial(link = "logit"))
anova(y.logit2, y.logit)

## Analysis of Deviance Table
##
## Model 1: y ~ x2
## Model 2: y ~ inc1000 + x2
##   Resid. Df Resid. Dev Df Deviance
## 1         18      22.563
## 2         17      21.081  1    1.4818
```

Assessing the weight of evidence using the bootstrap

- The full model contains all variables plus an intercept. We will denote its residual deviance by D_1 , and its approximate expected value will be $n - (p - 1)$
- The reduced model contains only $p_0 - 1$ of the predictor variables plus an intercept, we will denote its residual deviance by D_0 , and its approximate expected value is $n - (p_0 - 1)$ when the reduced model is the true model.
- The analysis of deviance statistic is

$$F^* = \frac{(D_0 - D_1)/(p - p_0)}{D_1/(n - (p - 1))}$$

which is approximately distributed as F on $p - p_0$ and $n - (p - 1)$ degrees of freedom

Parametric bootstrap to calculate a p-value

- Calculate D_0 , D_1 and F^* .
- Simulate R datasets according to the reduced model. Each dataset should have the same number of observations as the original dataset.
- For each simulated dataset, compute
 1. D_0 . Call it D_0^* .
 2. D_1 . Call it D_1^* .
 3. $F^{**} = \frac{(D_0^* - D_1^*) / (p - p_0)}{D_1^* / (n - (p - 1))}$
- Calculate the proportion of F^{**} values that exceed F^* . This is the bootstrap estimate of the p-value for the analysis of deviance test.

Inference on the regression coefficients

- The Fisher information matrix is the second derivative of the log likelihood.
- The second derivative is related to the curvature of the function.
 - If the likelihood function is flat near the parameter estimate, then the second derivative will be small, so its inverse will be large leading to a large variance.
 - If the likelihood function has a fairly sharp peak at the parameter estimate, then the second derivative will be large, so its inverse will be small leading to a small variance.
- Standard errors can be calculated for the parameter estimates given in the `glm` output.
- $\hat{\theta} - \theta \sim N(0, -J^{-1})$, where J is the Fisher information matrix. (Standard errors are square roots of the diagonal elements of the information matrix.)

Inference on the regression coefficients

AC1 Example

```
summary(y.glm0)$coefficients
```

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	1.39188135	0.2950282	4.7177911	2.384191e-06
##	x	-0.03833606	0.1384793	-0.2768359	7.819061e-01

The standard error column was computed from the square root of the diagonal of the inverse of the Fisher information matrix.

Residuals

- For a model in which $\mu_j = E[y_j]$, we could define the raw residuals as

$$y_j - \hat{\mu}_j.$$

Here, $\hat{\mu}_j$ is determined from $g^{-1}(\mathbf{X}_j\hat{\beta})$.

- Pearson residual:

$$r_j = \frac{y_j - \hat{\mu}_j}{\sqrt{\mathbf{Var}(\hat{\mu}_j)}}.$$

Residuals

- Deviance residual:

$$D = 2\phi(\ell_{\text{sat}} - \ell(\hat{\beta})).$$

$$D = \sum_{j=1}^n d_j.$$

Alternatively, we can think of d_j as

$$d_j = 2\phi(\ell_j(\theta_j) - \ell_j(\mathbf{X}_j\hat{\beta})).$$

$$\hat{\varepsilon}_j = \sqrt{d_j}\text{sign}(y_j - \hat{\mu}_j).$$

A bootstrap approach to model-checking

- Each term in the log likelihood sum should be reasonably large in the model.
- Any terms that are very small are indicators of observations that are failing to be modeled appropriately.
- The log likelihood sum could be very small if the fitted model does not correspond to where the data are.

A bootstrap approach to model-checking - simulation study

To demonstrate the effectiveness of the method we simulate two data sets containing 50 observations each.

- In the first case, the simulated response is Poisson with $\theta_j = 3 - .2x_j$, for a simulated covariate x
- The canonical link function is assumed, and a Poisson model is fit to the data.
- Using the fitted model, 10 data sets are simulated and the log likelihood contributions for each of the 50 x values are calculated.
- Side-by-side boxplots are computed for the resulting log likelihood contributions.

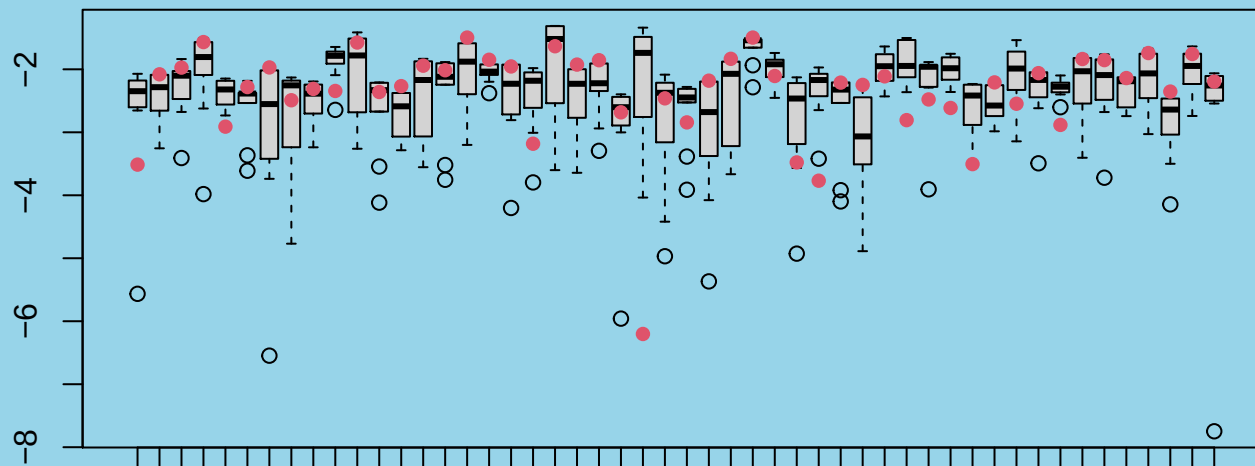
A bootstrap approach to model-checking - simulation study

```
n <- 50; x <- runif(n, min = 0, max = 10)
theta <- 3 - .2*x      ###
lambda <- exp(theta)
y <- rpois(n, lambda = lambda)
y.glm <- glm(y ~ x, family = poisson)
a <- coef(y.glm)[1]
b <- coef(y.glm)[2]
llikelihoodValues <- log(dpois(y, lambda = exp(a + b*x)))      ###
simloglike <- function(x, y.glm, N=10) {
  ll <- matrix(0, ncol=length(x), nrow=N)
  for (j in 1:N) {
    y <- simulate(y.glm)$sim_1
    sim.glm <- glm(y ~ x, family = poisson)
    a <- coef(sim.glm)[1]
    b <- coef(sim.glm)[2]
    loglikelihoodValues <- log(dpois(y, lambda = exp(a + b*x)))
    ll[j, ] <- loglikelihoodValues
  }
  ll <- data.frame(ll)  #names(ll) <- as.character(x)
  names(ll) <- ""
  ll
}
```

A bootstrap approach to model-checking

Distributions of observed likelihood contributions compared with resampled likelihood contributions. The model is correct in this case.

```
simlike <- simloglike(x, y.glm)
par(mar=c(1, 4, .1, .1))
boxplot(simlike, ylim=range(c(llikelihoodValues,
                             range(simlike))))
points(llikelihoodValues, pch=16, col=2)
```



A bootstrap approach to model-checking

- The red dots often appear near the top of the plot, which is where we would like them to be, for a well-fit model.
- we should expect a few - are not outside the range of the resampled log likelihood contributions.

A bootstrap approach to model-checking

- The second simulation was run with an incorrect model.
- the number of 0's has been inflated by multiplying the correct Poisson value by a random Bernoulli random variable.
 - When multiplied by 1, the value remains unchanged. item when multiplied by 0, the Poisson value may change to 0 from a nonzero value
 - 5% of the values are affected in this way.
- Compare the resulting boxplots from these two

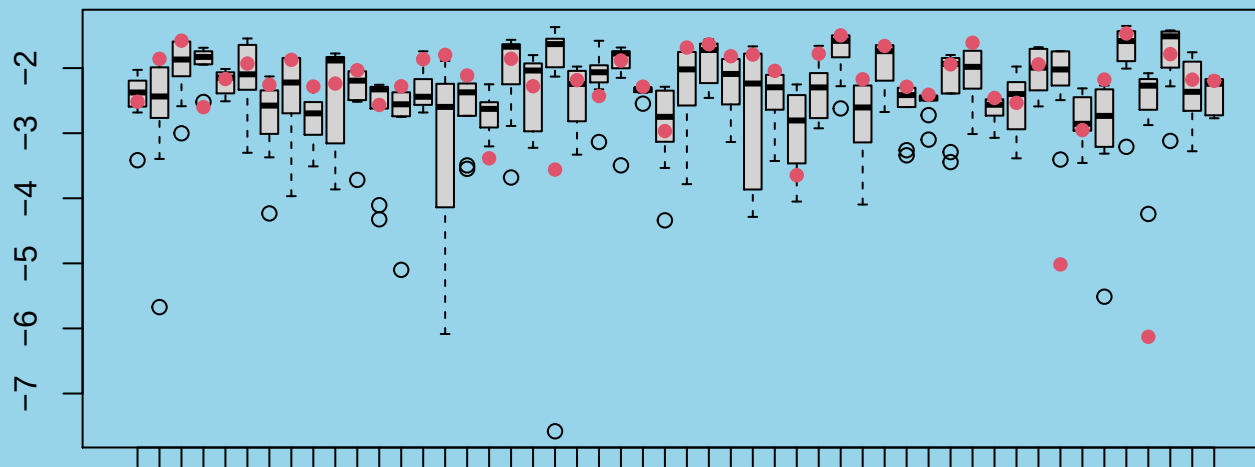
A bootstrap approach to model-checking

```
n <- 50
x <- runif(n, min = 0, max = 10)
theta <- 3 - .2*x
lambda <- exp(theta)
y <- (rpois(n, lambda = lambda))*rbinom(n, 1, prob=.95)
y.glm <- glm(y ~ x, family = poisson)
a <- coef(y.glm)[1]
b <- coef(y.glm)[2]
likelihoodValues <- log(dpois(y, lambda = exp(a + b*x)))
```

A bootstrap approach to model-checking

Distributions of observed likelihood contributions compared with resampled likelihood contributions. The model is incorrect in this case.

```
simlike <- simloglike(x, y.glm)
par(mar=c(1, 4, .1, .1))
boxplot(simlike, ylim=range(c(llikelihoodValues,
                             range(simlike))))
points(llikelihoodValues, pch=16, col=2)
```



A bootstrap approach to model-checking - AC1 data

We can apply this bootstrap method to any of our fitted GLM models.

Here, we check the appropriateness of the Poisson model for the AC1 data.

```
AC.glm <- glm(y ~ x, family = poisson, data = AC1)
a <- coef(AC.glm)[1]
b <- coef(AC.glm)[2]
likelihoodValues <- with(AC1,
  log(dpois(y, lambda = exp(a + b*x))))
```

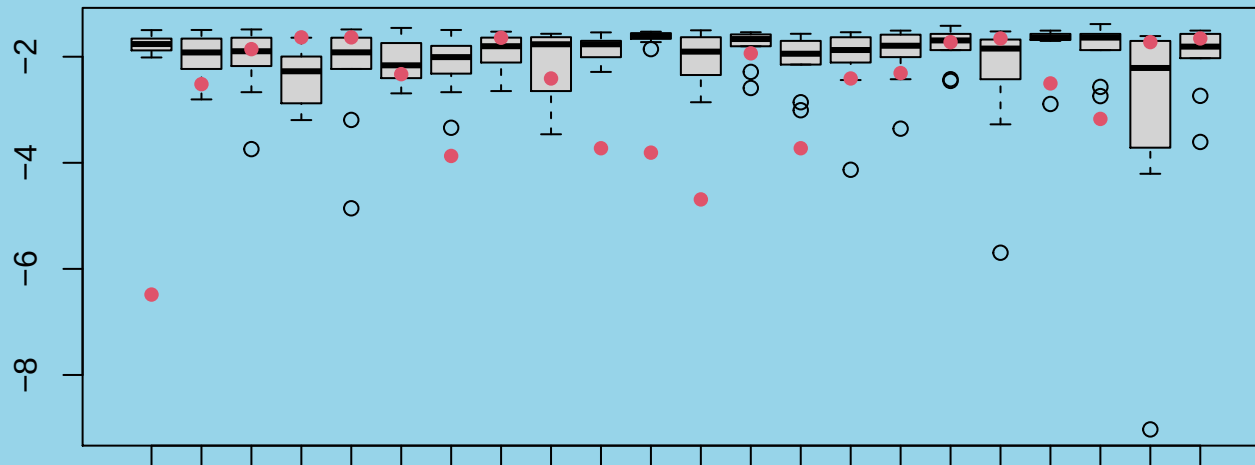
A bootstrap approach to model-checking - AC1 data

The following function simulates contributions to the log likelihood and can be used in a parametric bootstrap to assess the “likelihood” that the model is appropriate.

```
simloglike <- function(x, y.glm, N=10) {
  ll <- matrix(0, ncol=length(x), nrow=N)
  for (j in 1:N) {
    y <- simulate(y.glm)$sim_1
    sim.glm <- glm(y ~ x, family = poisson)
    a <- coef(sim.glm)[1]
    b <- coef(sim.glm)[2]
    loglikelihoodValues <- log(dpois(y, lambda =
      exp(a + b*x)))
    ll[j, ] <- loglikelihoodValues
  }
  ll <- data.frame(ll) #names(ll) <- as.character(x)
  names(ll) <- ""
  ll
}
```

A Bootstrap approach to model-checking - AC1 data

```
AClike <- with(AC1, simloglike(x, AC.glm))
par(mar=c(1, 4, .1, .1))
boxplot(AClike, ylim=range(c(llikelihoodValues, AClike)))
points(llikelihoodValues, pch=16, col=2)
```



Distributions of observed likelihood contributions compared with resampled likelihood contributions. This is a check on the Poisson regression model for the AC1 data. Some extreme low outliers indicate a poor fit.

Overdispersion and quasilikelihood

- The binomial and Poisson assumptions can be very restrictive: the mean must equal the variance in the Poisson case, while the mean and variance of the Bernoulli distribution are both related through p .
- Often, the variance will be much larger than the mean in count data; this is called **overdispersion**.
 - Clustering often leads to overdispersion.
- Quasi-likelihood methods have been developed to handle this problem. In `glm()`, use `family=quasipoisson`. (This can be employed to handle the AC1 data set, for example.)
- Hypothesis testing results using the F distribution remain approximately true if large sample is used.

A slightly more complicated dataset

The `lesions` data were collected at the University of Manitoba: precancerous lesions in the colons of 66 rats:

```
source("lesions.R")
```

```
summary(lesions)
```

```
##      T              INJ              SECT              RAT
##  1:132    Min.      :1.000    1:66    1          :54
##  2:144    1st Qu.:1.000    2:66    2          :54
##  3:120    Median :2.000    3:66    3          :54
##              Mean  :2.333    4:66    4          :54
##              3rd Qu.:4.000    5:66    5          :54
##              Max.   :4.000    6:66    6          :54
##                                (Other):72
##      ACF.Total      ACF.total.mult      id
##  Min.      :  0.00    Min.      :  0.0    Min.      :  1.0
##  1st Qu.: 14.00    1st Qu.: 31.0    1st Qu.:17.0
##  Median : 28.00    Median : 65.5    Median :33.5
##  Mean    : 38.23    Mean    :105.4    Mean    :33.5
##  3rd Qu.: 53.25    3rd Qu.:140.8    3rd Qu.:50.0
##  Max.    :213.00    Max.    :719.0    Max.    :66.0
##
```

Lesions in rat colons

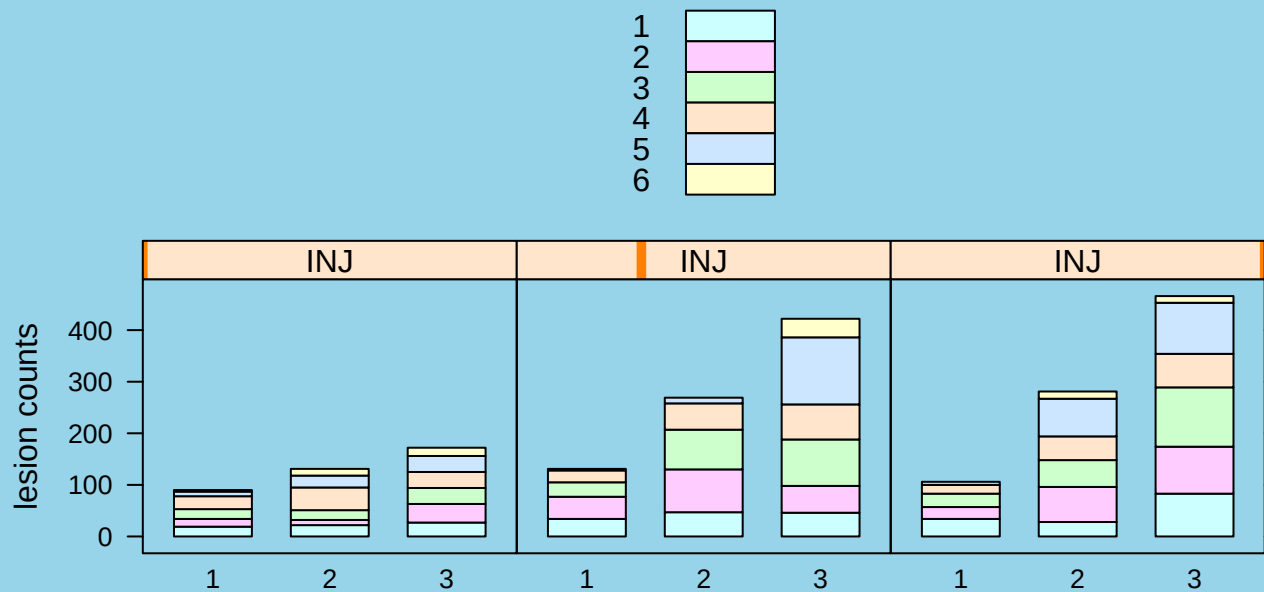
The rats had been subjected to differing numbers of injections of carcinogen (INJ: 1, 2 or 4 doses) and sacrificed at 6, 12 or 18 weeks (T)

Counts of lesions (called ACF) were taken on the colon walls, divided up into 6 sections (SECT) beginning at the proximal end and ending at the distal end.

Interest centers on how the numbers and area of the lesions grow with time and number of doses, and in the different regions of the colon.

Counting lesions in some of the rats

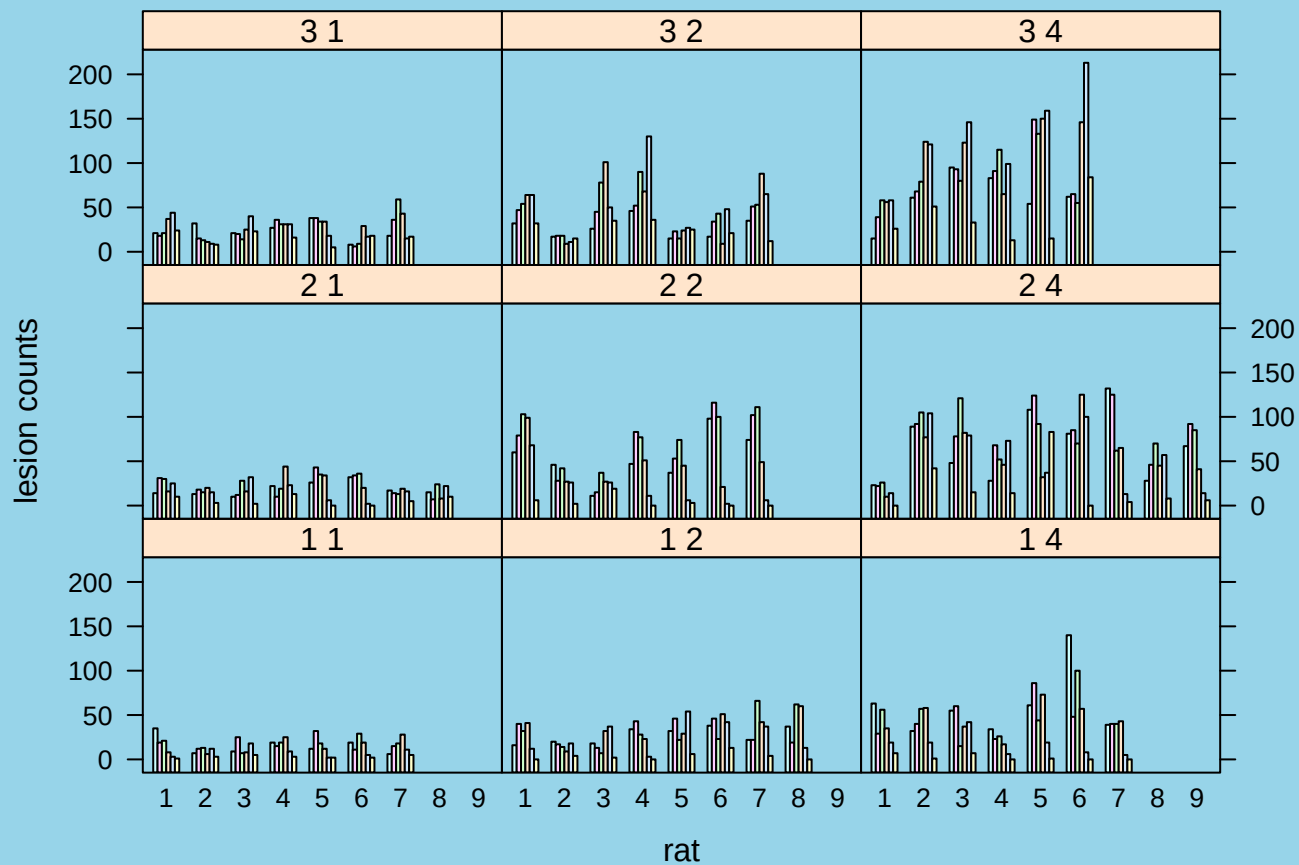
```
barchart(ACF.Total ~ T|INJ, groups=SECT, data =  
  subset(lesions, RAT==4), stack=TRUE, auto.key=TRUE,  
  ylab="lesion counts")
```



Section 5 has a lot of lesions. And it seems that the number of lesions does not change from dose level 2 to dose level 4.

Viewing the counts in all rats

```
barchart (ACF.Total ~ RAT|paste(T, INJ), groups=SECT,  
data = lesions, stack=FALSE, ylab="lesion counts",  
xlab = "rat")
```



Modelling the ACF data

We start with modelling the total number of ACF as a function of time and injection, for each rat. (That is, we will sum over all colon sections in each rat.)

```
ACF.All <- aggregate(ACF.Total ~ id + INJ + T,
  FUN=sum, data = lesions)
```

```
summary(ACF.All)
```

##	id	INJ	T	ACF.Total
##	Min. : 1.00	Min. :1.000	1:22	Min. : 53.0
##	1st Qu.:17.25	1st Qu.:1.000	2:24	1st Qu.:106.8
##	Median :33.50	Median :2.000	3:20	Median :180.0
##	Mean :33.50	Mean :2.333		Mean :229.4
##	3rd Qu.:49.75	3rd Qu.:4.000		3rd Qu.:304.8
##	Max. :66.00	Max. :4.000		Max. :660.0

Modelling the ACF data - Poisson

A Poisson GLM is a reasonable first guess for such data. We include an interaction term for injection and time effects:

```
ACF.poisson <- glm(ACF.Total ~ INJ * T, data = ACF.All, family=poisson)
summary(ACF.poisson)
```

```
##
## Call:
## glm(formula = ACF.Total ~ INJ * T, family = poisson, data = ACF.All)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -17.2309   -4.2434   -0.9751    4.2298   13.0646
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   4.30032    0.04199 102.423  < 2e-16 ***
## INJ           0.28459    0.01386  20.530  < 2e-16 ***
## T2            0.39760    0.05320   7.474 7.80e-14 ***
## T3            0.34349    0.05344   6.427 1.30e-10 ***
## INJ:T2        0.02167    0.01724   1.257   0.209
## INJ:T3        0.11758    0.01741   6.754 1.44e-11 ***
## ---
```

```
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 5894.4  on 65  degrees of freedom
## Residual deviance: 2049.2  on 60  degrees of freedom
## AIC: 2528
##
## Number of Fisher Scoring iterations: 4
```

Note that the residual deviance is much larger than its degrees of freedom.

This is a strong hint that the model is very inaccurate.

Thus, we cannot trust the p-values for the effects at all.

Modelling the ACF data - quasipoisson

The simplest alternative is to use the `quasipoisson` family instead of `poisson`.

This relaxes the Poisson assumption and allows us to handle overdispersion.

The model is no longer interpretable in the same way, however, since it is not based on a real probability model.

Modelling the ACF data - quasipoisson

```
ACF.qp <- glm(ACF.Total ~ INJ * T, data = ACF.All, family=quasipoisson)
summary(ACF.qp)
```

```
##
## Call:
## glm(formula = ACF.Total ~ INJ * T, family = quasipoisson, data = ACF.A
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -17.2309   -4.2434   -0.9751    4.2298   13.0646
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   4.30032    0.24178  17.786 < 2e-16 ***
## INJ           0.28459    0.07983   3.565 0.000721 ***
## T2            0.39760    0.30635   1.298 0.199306
## T3            0.34349    0.30775   1.116 0.268813
## INJ:T2        0.02167    0.09928   0.218 0.827981
## INJ:T3        0.11758    0.10026   1.173 0.245511
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 33.1603)
##
```

```
##      Null deviance: 5894.4  on 65  degrees of freedom
## Residual deviance: 2049.2  on 60  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 4
```

One thing that this quasiliikelihood approach tells us is that the interaction effect is not significant.

Fit the model again, without the interaction term.

Note the effect on the `INJ` and `T` coefficients.

The standard errors on the coefficients are smaller than with the interaction term.

Because of the multiple testing (2 tests), we should demand that the p-values be less than about .025 in order to say that we have strong evidence of effects.

Modelling the ACF data - quasipoisson

The required code to omit the interaction term is as follows.

```
ACF.qp <- glm(ACF.Total ~ INJ + T, data = ACF.All, family=quasipoisson)
summary(ACF.qp)
```

```
##
## Call:
## glm(formula = ACF.Total ~ INJ + T, family = quasipoisson, data = ACF.A
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -17.7800   -4.1140   -0.6099    3.6721   13.5608
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   4.14947    0.14540  28.538 < 2e-16 ***
## INJ           0.33844    0.03733   9.066 5.72e-13 ***
## T2            0.45345    0.12422   3.651 0.000539 ***
## T3            0.67487    0.12467   5.413 1.06e-06 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 32.98798)
```

```
##  
##      Null deviance: 5894.4  on 65  degrees of freedom  
## Residual deviance: 2110.9  on 62  degrees of freedom  
## AIC: NA  
##  
## Number of Fisher Scoring iterations: 4
```

Modelling the ACF data - negative binomial GLM

An alternative to quasilielihood is the negative binomial model.

This is a model for counts which allows for overdispersion.

In fact, the distribution arises from a form of clustering.

The *MASS* package has a function that fits generalized linear models, based on the negative binomial distribution.

Modelling the ACF data - negative binomial GLM

```
library(MASS)
ACF.nb <- glm.nb(ACF.Total ~ INJ + T, data = ACF.All)
summary(ACF.nb)

##
## Call:
## glm.nb(formula = ACF.Total ~ INJ + T, data = ACF.All, init.theta = 7.1
##      link = log)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -3.0685  -0.7880  -0.0612   0.4765   2.2565
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   4.12938    0.11996  34.424  < 2e-16 ***
## INJ           0.35607    0.03751   9.493  < 2e-16 ***
## T2            0.43902    0.11259   3.899  9.65e-05 ***
## T3            0.61777    0.11766   5.251  1.52e-07 ***
## ---
## Signif. codes:
##  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(7.19) family taken to be 1
##
```

```
##      Null deviance: 184.98  on 65  degrees of freedom
## Residual deviance:  67.16  on 62  degrees of freedom
## AIC: 763.56
##
## Number of Fisher Scoring iterations: 1
##
##
##              Theta:  7.19
##            Std. Err.:  1.27
##
## 2 x log-likelihood: -753.564
```

Look at the residual deviance value.

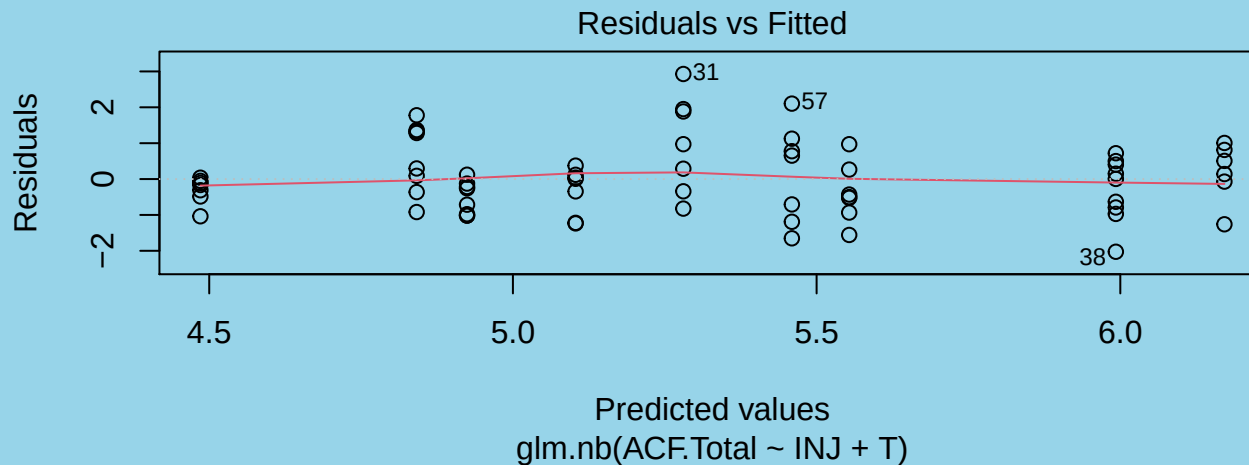
The residual deviance is in line with the residual degrees of freedom, supporting the model – of course, we cannot really conclude much about the model until we examine graphical diagnostics.

If the model is appropriate, we can now conclude quite safely that the time and injection effects are significant.

We can inspect the residual plot produced with the following code.

Modelling the ACF data - negative binomial GLM

```
plot(ACF.nb, which=1)
```



No real trend in the mean residual level is apparent, but there is a pattern to the variation in the residuals which would have to be examined more closely, say, by simulating from this model. Observation 38 is being flagged as an outlier.

Analyzing the section effects requires more sophisticated machinery

If we want to study an effect due to colon section, we need to use a mixed-effects model.

You may want to omit the final part of the last section of Chapter 8 until you have read Chapter 10.

But do read it eventually!